

2. Architecture et correspondance technologique

Proposition de projet — Enterprise Inventory & Logistics Management System (EILMS)

Partie du document : Section 2 — Architecture & Tech Stack Mapping

Ce tableau relie chaque **couche** de l'architecture du projet aux **concepts du cours** qu'elle est censée mobiliser. C'est une carte de route : chaque étape du projet va piocher dans une de ces couches.

Couche	Concepts couverts
Core Layer	Interfaces, classes abstraites, polymorphisme, exceptions personnalisées, expressions lambda, Stream API, multithreading
Persistence & Web	JDBC, Servlets, MVC, JPA/Hibernate, services REST, Spring Core, Spring Data JPA, Spring Security
DevOps & Conteneurisation	Docker, builds multi-étapes, déploiements Kubernetes, Services, ConfigMaps
Déploiement & Serveurs	Apache Tomcat, GlassFish, synchronisation de pool de threads

Explication détaillée par couche

Core Layer : c'est ce qu'on a déjà vu au Step 1 — les bases orientées objet du langage. Exemple : une interface `InventoryItem` définit un contrat (« tout item a un id, un nom, une quantité »), et une classe abstraite `AbstractWarehouseItem` peut fournir une implémentation partielle commune à plusieurs types d'items.

Persistence & Web : c'est la suite du Step 2 — on passe du JDBC brut à Spring Data JPA (qui génère automatiquement les requêtes), et on expose les données via des services REST plutôt qu'en console.

DevOps & Conteneurisation : une fois l'application fonctionnelle, on la « boîte » dans une image Docker pour qu'elle tourne partout de façon identique, puis Kubernetes orchestre plusieurs copies de cette boîte automatiquement.

Déploiement & Serveurs : test de l'application sur deux serveurs différents (Tomcat = léger, seulement des servlets ; GlassFish = serveur Jakarta EE complet), pour comprendre les différences de comportement et de performance.